

Licence 3 Informatique

Année Universitaire 2025 – 2026

???

Période du stage :

du 13 avril au 28 août 2026

Rédigé et présenté par :
Stella-Rose MILLE

Tuteur en Entreprise :
Federico LOCHBAUM

Tuteur Universitaire :
Romain ROUVOY

Table des matières

1	Contexte du Stage	2
1.1	Présentation du Laboratoire et de l'Équipe	2
1.2	Contexte de l'Application	2
2	Description d'une réalisation technique majeure	3
2.1	Contexte et objectif du shrinking	3
2.2	Choix techniques et justifications	3
2.3	Développements et travaux réalisés	4
2.4	Difficultés rencontrées et solutions apportées	4
3	Conclusion et Bilan du Stage	5
3.1	Adéquation de la Réalisation aux Objectifs et Perspectives	5
3.2	Présentation du Travail à Réaliser pour la Suite	5
3.3	Enseignements et Apports du Stage	5
3.4	Remerciements	5

1. Contexte du Stage

1.1 Présentation du Laboratoire et de l'Équipe

Ce stage se déroule au sein d'INRIA (Institut National de Recherche en Sciences et Technologies du Numérique), un établissement de recherche dédié aux sciences du numérique. J'y ai intégré l'équipe de recherche Evref, une équipe dont le principal objectif est l'étude de l'évolution des grands systèmes informatiques. Leurs travaux s'articulent autour de trois axes : L'analyse et la migration de système existants, le développement d'outils pour supporter l'évolution, l'infrastructure permettant de supporter l'évolution logiciel.

1.2 Contexte de l'Application

Mon travail s'inscrit dans le cadre d'un projet de thèse de l'équipe, centré sur le développement d'Ume : un framework de Property-Based Testing (PBT) conçu spécifiquement pour l'écosystème Pharo (un langage purement orienté objet). Contrairement aux tests unitaires classiques où le développeur écrit manuellement les inputs, le PBT génère automatiquement des centaines d'inputs aléatoires pour valider le bon fonctionnement du logiciel. Ume cherche à spécialiser ce concept dans la recherche de problèmes de performances.

AJOUTER DES DETAILS

c'est quoi une campagne ???

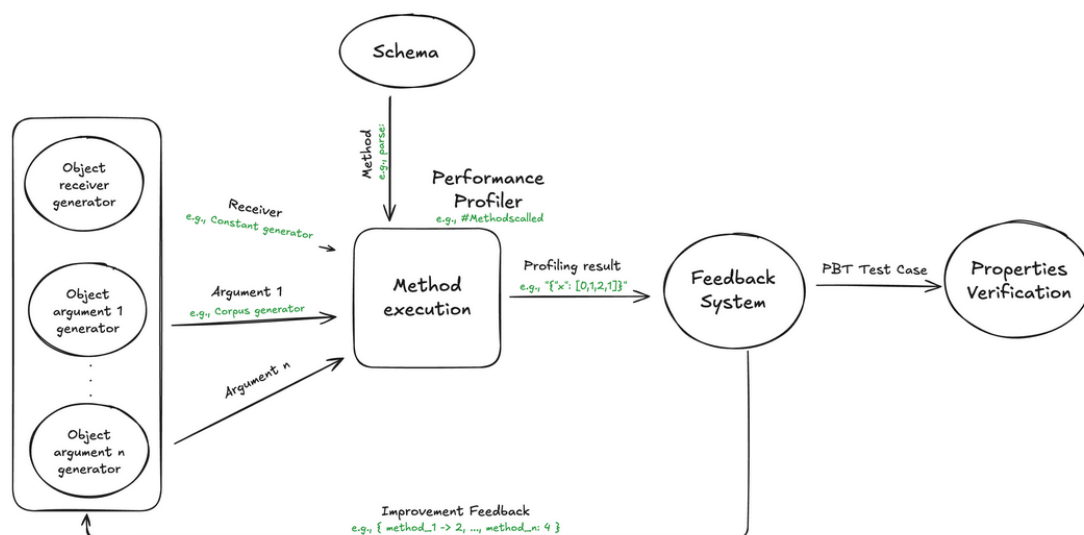


FIGURE 1.1 – Architecture d'Ume

mettre ce schéma à ma sauce en highlightant les sections sur les quelles je travaille

2. Description d'une réalisation technique majeure

Dans cette section, je vais présenter le shrinking, pourquoi ce concept a été ajouté à Ume et comment ça fonctionne.

2.1 Contexte et objectif du shrinking

Le shrinking peut être utile lorsqu'un test échoue sur un input complexe générée durant une campagne. L'objectif est de réduire cet input à une taille minimale reproduisant la même erreur.

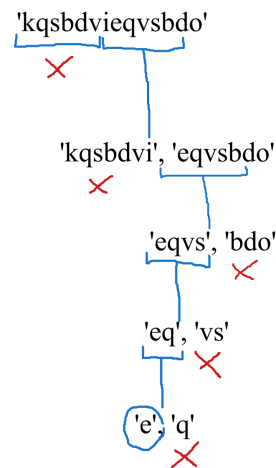


FIGURE 2.1 – exemple de shrinking sur une chaîne

Bien que l'utilité de l'application du shrinking à des problèmes de performances est une question que l'on se pose, cette fonctionnalité reste indispensable d'un point de vue utilisateur pour permettre la localisation précise du problème et simplifier la phase de debuggage.

2.2 Choix techniques et justifications

Afin d'implémenter le shrinking dans Ume, je me suis basé sur les algorithmes présentés dans le Fuzzing Book ¹.

Pour débiter, j'ai implémenté un algorithme de Delta Debugging classique qui permet de traiter la réduction de chaînes. Cet algorithme est basé sur le principe du "diviser pour régner", il segmente les chaînes de caractères et évalue chaque morceau afin d'isoler la faille. Malgré son efficacité pour traiter des chaînes, le Delta Debugging montre ses limites au moment de traiter des grammaires complexes. Une simple segmentation casse la syntaxe et les tests échouent pour des erreurs de parsing.

C'est pourquoi il est nécessaire de coupler cette méthode à des réducteurs basés sur la grammaire, afin de garantir que chaque entrée soit syntaxiquement valide.

1. <https://www.fuzzingbook.org/html/Reducer.html#Delta-Debugging>

2.3 Développements et travaux réalisés

Les algorithmes restant complexes, j’ai appliqué la méthodologie du test-driven development (TDD). Cela m’a permis de définir le comportement attendu avant même de commencer le code. J’ai implémenter deux algorithme de réduction avec un design pattern Strategy : la réduction par sous arbres et par expansion alterné. Cette méthodologie permetras d’implémenter facilement l’ajout de futurs algorithmes.

2.4 Difficultés rencontrées et solutions apportées

Durant mon stage, j’ai rencontrés plusieurs difficulté, spécifiquement sur l’implémentation du shrinking, telle que :

- Adaptation Smalltalk/Pharo : Les exemples d’algorithmes du Fuzzing Book sont écrits en Python. Adapthé des algorithmes en Python vers Pharo a demandé uune compréhension complète de la logique mais aussi une réflexion sur les structure de données.
- Solidité de l’exploration : Garantir que le shrinker explore de manière efficace les branches de réduction a nécessité a nécessité de nombreux tests. (à compléter)

3. Conclusion et Bilan du Stage

3.1 Adéquation de la Réalisation aux Objectifs et Perspectives

Les objectifs de cette première moitié de stage ont été atteints : l'API d'Ume a été simplifiée pour l'utilisateur final et un début de shrinker fonctionnelle respectant la grammaire a été intégrée.

Concernant la question de l'évaluation, la mise en place de benchmarks constitue l'étape logique suivante. Ils permettront de comparer différents algorithmes de shrinking.

3.2 Présentation du Travail à Réaliser pour la Suite

Mon stage s'étendant jusque fin août, j'ai plusieurs missions pour la suite, le shrinker doit continuer à être développé, notamment par l'implémentation d'autres algorithmes de réductions et par la comparaison entre eux.

Afin d'avoir une lecture plus simple des résultats d'une campagne, l'ajout d'un feedback visuel est prévue. Ce retour permettra d'avoir une vue d'ensemble sur la campagne et d'en recueillir les informations facilement.

L'ajout d'un plugin pour Dr Test (une UI plugin-based permettant de gérer les tests) est aussi prévue.

3.3 Enseignements et Apports du Stage

Ce stage m'a permis de développer une forte autonomie. Dès les premiers jours, mon tuteur a mis l'accent sur l'organisation et l'autonomie. Nous avons structuré notre flux de travail avec GitHub pour la gestion de versions (apprentissage des Pull Requests bien documentées) et Obsidian pour la documentation de mes tâches quotidiennes.

Sur le plan technique, l'utilisation rigoureuse du TDD m'a permis de me canaliser lorsque je perdais de vue l'objectif final, tandis que l'application concrète des Design Patterns a renforcé mes compétences en conception logicielle.

3.4 Remerciements

Je souhaite remercier mon tuteur d'entreprise, Federico Lochbaum, pour ses précieux conseils, sa patience et sa confiance. Mon tuteur universitaire, Romain Rouvoy, pour son suivi rigoureux. Enfin, j'adresse mes remerciements à Imen Sayar pour son soutien continu et pour avoir appuyé ma candidature pour ce stage.